SIMATS ENGINEERING

CO5-ASSESSMENT TOOL -3

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

Student Name: R.GAUTHAM KRISHNA **Reg. No.: 192421128**

Course Code: **CSA0613** **Slot:** A

Course Name: **DESIGN AND ANALYSIS OF ALGORITHMS**

Course Faculty: Dr. Mary Harin Fernandez F

EVALUATION SCORE REPORT

EVALUATION SCORE

Criterion	Weightage	Level	Marks Obtained
Technical Knowledge	25	Level 4	24/25
Problem Solving	25	Level 4	22/25
Analytical Thinking	15	Level 4	14/15
Communication	15	Level 4	14/15
Confidence	10	Level 4	9/10
Response to Questions	10	Level 4	10/10
TOTAL	**100**	**Level 4**	**93/100**

FINAL SCORE: 93/100

OVERALL PERFORMANCE LEVEL: LEVEL 4 – EXCELLENT

1. What is Backtracking and how does it work?

Answer:
Backtracking is an algorithmic technique used to solve problems by trying possible solutions step by step. If a selected choice leads to an invalid solution, the algorithm goes back to the previous step and tries another choice.

It is commonly used in problems such as N-Queens, Sudoku, Graph Coloring, and Hamiltonian Cycle. Backtracking reduces unnecessary searching by rejecting invalid paths early.

2. What is a State-Space Tree?

Answer:
A State-Space Tree is a tree representation of all possible solutions to a problem.

- **The root represents the initial state.**
- **Each node represents a partial solution.**
- **Each branch represents a possible choice.**
- **A leaf node represents a complete solution or failure.**

Backtracking explores this tree and returns when a branch cannot produce a valid solution.

3. Explain the N-Queens problem using Backtracking.

Answer:
The N-Queens problem requires placing N queens on an N × N chessboard so that no two queens attack each other.

Using Backtracking, we place one queen in each row and check whether its position is safe. We must ensure that no other queen is in the same:

- **Column**
- **Main diagonal**

- **Secondary diagonal**

If a position is invalid, we try another position. If no position works, we go back to the previous row.

4. What is Pruning in Backtracking?

Answer:
Pruning means stopping the exploration of a path that cannot lead to a valid solution.

For example, in the N-Queens problem, if two queens attack each other, there is no need to continue exploring that arrangement.

Pruning improves efficiency because the algorithm avoids checking unnecessary possibilities. Bounding functions can also help determine whether a partial solution is worth exploring.

5. How does the Subset Sum problem work?

Answer:
The Subset Sum problem checks whether a subset of given numbers has a sum equal to a target value.

For every element, we have two choices:

1. **Include the element.**
2. **Exclude the element.**

For example:

Set = {3, 4, 5, 2}
Target = 9

A valid subset is {4, 5} because:

4 + 5 = 9

Backtracking tries different combinations until it finds a solution or determines that no solution exists.

6. What is the Hamiltonian Cycle problem?

Answer:
A Hamiltonian Cycle is a cycle in a graph that visits every vertex exactly once and returns to the starting vertex.

Backtracking starts from one vertex and tries to add connected vertices to the path. Before adding a vertex, the algorithm checks that:

- **The vertex is connected to the previous vertex.**
- **The vertex has not already been visited.**

After visiting all vertices, it checks whether the last vertex connects back to the starting vertex.

7. Explain Graph Coloring using Backtracking.

Answer:
Graph Coloring means assigning colors to graph vertices so that no two connected vertices have the same color.

The Backtracking algorithm assigns a color to one vertex at a time. Before assigning a color, it checks whether any adjacent vertex already has that color.

If no valid color is available, the algorithm goes back and changes the color of a previous vertex.

Graph Coloring is used in scheduling, timetables, frequency allocation, and compiler design.

8. How does a Sudoku Solver use Backtracking?

Answer:
A Sudoku solver first finds an empty cell and tries numbers from 1 to 9.

Before placing a number, it checks whether the number already exists in the:

- **Same row**
- **Same column**
- **Same 3 × 3 box**

If the number is valid, the algorithm continues to the next empty cell. If it later reaches an impossible situation, it removes the previous number and tries another one.

This process continues until the Sudoku is solved.

9. Explain P, NP, NP-Hard, and NP-Complete.

Answer:

- **P: Problems that can be solved in polynomial time.**
- **NP: Problems whose solutions can be verified in polynomial time.**
- **NP-Hard: Problems that are at least as difficult as NP problems.**
- **NP-Complete: Problems that are both NP and NP-Hard.**

Examples of NP-Complete problems include Subset Sum, Hamiltonian Cycle, and certain Graph Coloring decision problems.

These concepts are important for understanding how difficult computational problems are.

10. What is the difference between Exact, Approximation, Greedy, and Heuristic algorithms?

Answer:

Exact Algorithm

Always finds the optimal solution, but may require more time.

Approximation Algorithm

Finds a solution close to the optimal solution with a known quality guarantee for applicable problems.

Greedy Algorithm

Makes the best local choice at each step, hoping to produce a good or optimal global solution.

Heuristic Algorithm

Uses practical strategies to find good solutions quickly, but does not necessarily guarantee the optimal solution.

For large real-world problems such as route planning, scheduling, and resource allocation, approximation or heuristic approaches are often preferred because exact algorithms may take too much time.